Your TA, Explained

DSIC-2907 • A plain-English research and ownership guide

Prepared from the project repository and saved research artifacts

Evidence snapshot: 3 October 2026

Find your way around

1. Start here: the whole TA in five minutes 3
2. Objective, motivation, and contribution 4
3. Research questions and what would count as success 5
4. Current state: what exists and what does not 6
5. The existing results, explained honestly 7
6. Understand the data and protect the experiment 8
7. How the planned federated system works 9
8. Experimental infrastructure and RunPod ownership 11
9. Demand and the two controls 15
10. B0–B3: the experiment that isolates the idea 17
11. Measurement: bytes, time, quality, and fairness 18
12. Analysis and claims you can defend 19
13. Future plan: a practical route to completion 20
14. Open decisions and immediate priorities 22
15. Repository ownership and safe operation 23
16. Taking ownership: explain it, question it, defend it 25
17. Source map and verification record 27
18. Plain-English glossary 30

Infrastructure-planning revision: 5 October 2026. The historical evidence snapshot remains 3 October 2026. Section 8 adds a proposed RunPod testbed and ownership checklist; it does not report a deployed host, newly completed FL work, or a new audit of the research implementation.

Day 2 implementation revision: 6 October 2026. Ten-client Dirichlet partitions at alpha 0.5 are frozen for seeds 42, 123, and 2026. At that revision, asynchronous FL and B0–B3 comparisons remained planned. The 3 October baseline results and verification record remain historical.

Day 3 implementation revision: 6 October 2026. B0 asynchronous CPU training now has a completed ten-client, 30-update pilot with exact arrival replay. Current status statements below include this evidence. The official test was not evaluated, and main B0–B3 comparisons, transport accounting, and protocol freeze remain planned.

1. Start here: the whole TA in five minutes

Your TA asks whether a shared network-attack detection model can learn with less communication by adapting who contributes and how much each client sends. The decisions depend on each client's observed workload and current condition.

The full working article title is *Demand-Aware Adaptive Participation and Update Budgeting for Communication-Efficient Asynchronous Federated Network Intrusion Detection under Non-IID Data*. The long title contains the experiment's boundaries; you do not need to use it every time you explain the idea.

Imagine ten branches of an organization. Each branch has network-traffic records. Some branches have more examples, different proportions of attacks, or slower connections. They train copies of the same model and send model changes to a central server. Your research asks whether giving every branch the same communication treatment wastes resources.

The proposed method observes each branch, estimates its needs, and adjusts two things: **its opportunity to contribute** and **the amount of its model update that it sends**. It must save communication without an unacceptable loss in detection quality.

The most important status fact

Days 1–3 are complete through the B0 pilot. The main federated-learning comparison remains planned. The completed work prepares the dataset, trains one centralized model, measures its predictions, and saves traceable artifacts. Ten-client partitions cover train-fit exactly once for three seeds. A seed-42 B0 pilot applies 30 updates asynchronously with fixed local training and full budget. Demand estimation, adaptive scheduling, compression, transport accounting, and B0–B3 comparisons remain future work.

Question	Answer today
Is there a clear research objective?	Yes: reduce communication while protecting detection quality.
Is the data/model foundation working?	Yes: one recorded centralized run and substantive tests exist.
Has the proposed method been shown to work?	No: a B0 pilot exists, but joint demand control and paired B0–B3 comparisons remain planned.
Is the implementation ready for deployment?	No: this is a controlled research prototype.
What should happen next?	Complete Day 4 controlled versioning/staleness verification on the B0 backbone.

Question	Answer today
Must B3 win for the TA to be valid?	No: a well-controlled negative result is still a research result.

How to read this guide

Read Sections 1–5 for the purpose and honest current state. Read Sections 6–12 to understand the method, proposed infrastructure, and experiment. Use Sections 13–17 to manage the work, explain it to a supervisor, and locate evidence. Section 18 is a quick glossary.

Evidence labels: *Implemented* means source code exists; *recorded result* means an archived run produced it; *planned* means the repository describes it but has not demonstrated it; *open decision* means it still needs a precise rule. An example is not a research result. **TBD** means a value or procedure has not yet been chosen or verified.

Sources: [S1–S6]. This guide describes the inspected working tree, including existing uncommitted documentation edits. It is not a new literature review or an approval of unresolved protocol choices.

2. Objective, motivation, and contribution

The practical problem

A Network Intrusion Detection System, or NIDS, tries to distinguish normal traffic from attacks. In this TA, it learns from existing labeled records. It does not yet inspect live traffic or block attacks.

Federated Learning, or FL, trains a shared model using separate clients. In the intended experiment, raw training examples stay with each logical client during local training, while model information moves between client and server. Repeated model transfers can be costly, especially when client connections and workloads differ.

Asynchronous FL lets the server apply an arriving update without waiting for every other client. That removes the requirement to finish together, but introduces its own problem: a slow client's update may come from an older model. Faster clients may also contribute much more often.

Your proposed controller uses current client conditions to make communication decisions. The research is about whether those decisions improve the balance between communication cost, elapsed time, and model quality.

The objective in one sentence

> Determine whether observable client state can guide participation and update size so that asynchronous FL reaches useful intrusion-detection quality with fewer transmitted bytes, while keeping quality within a predefined tolerance.

What "resource allocation" means here

The DSIC29-07 catalog discusses choosing resource configurations based on workload, including CPU and memory. This TA adapts that principle to **communication resources**: opportunities to participate and the amount of update information sent. It is a conceptual adaptation of the catalog, not a literal CPU/memory allocation experiment.

Local epochs, batch size, model, numerical precision, and compute limits must stay the same across the four policies. Otherwise, an improvement might come from giving one policy more computation instead of making better communication decisions.

What the contribution can reasonably be

1. A clear mapping from observable client state to two demand scores.
2. Joint control of participation and update budget on one shared asynchronous backbone.
3. A four-policy experiment that separates each control's effect.
4. Actual serialized-byte measurements, quality safeguards, and reproducible evidence.

Asynchronous FL, client selection, and sparse model updates are not individually claimed as new. The supervisor's audit explicitly warns against that framing. A "first-ever" claim would require a separate, current literature review; this guide does not establish one.

Ownership test: you should be able to say which two things change, which things stay fixed, and why that makes the comparison meaningful.

Sources: [S1], [S2], [S9], [S10].

3. Research questions and what would count as success

The questions you need to answer

Question	Plain-English meaning	Evidence needed
RQ1: How should state become demand?	How do we turn observations into useful decisions?	Defined scores, normalization, thresholds, and replayable actions.
RQ2: Does joint control help?	Does B3 use fewer bytes without harming quality too much?	Paired B3/B0 results, time, quality, and uncertainty.
RQ3: Which control helps?	Is the benefit from participation, update size, or both?	B1 and B2 comparisons as well as B3.
RQ4: When does it help or fail?	What happens with unequal data and unequal client/network conditions?	State/action logs, stress runs, and failure analysis.
RQ5: Does it generalize?	Does the pattern survive another dataset or physical testbed?	Optional later replication; not required for the core month.

The planned practical targets

The research documents propose the following criteria. They are **targets to test, not achievements**:

- At least **15% less communication** for the predefined comparison endpoint.
- The lower end of the 95% confidence interval for final Macro-F1 difference, B3 minus B0, must be **above −0.01**.
- Time to the quality target should worsen by **no more than 10%**.
- Communication savings must remain after indices, headers, and metadata are counted.
- No persistent client starvation; B3 should offer a useful trade-off against the individual controls.

One percentage point is an absolute difference of 0.01 on a zero-to-one metric. For example, 0.84 to 0.83 is a one-percentage-point decline. It is not the same as a one-percent relative change.

The documents emphasize both total bytes and bytes-to-target. **Before main runs, name the primary communication endpoint explicitly.** A policy can send fewer total bytes simply because it makes less progress; comparing the bytes needed to reach the same quality helps expose that.

How to judge possible outcomes

Illustrative outcome	Honest interpretation
Fewer bytes, acceptable quality, acceptable time	Evidence supporting practical usefulness.
Fewer bytes, large quality loss	Communication improved, but the quality condition failed.
Good average quality, wide uncertainty	Evidence is inconclusive about non-inferiority.
B2 matches B3	The joint mechanism has not shown clear added value.
Gains only with slow/unequal networks	A conditional benefit, not a universal benefit.
B3 loses under controlled conditions	A valid finding; investigate the mechanism and report it.

The exact Macro-F1 target for "to-target" comparisons is still open. Do not take the centralized test score and silently use it as the FL target.

Sources: [S1, Sections 7–8 and 23–24], [S2, Days 14 and 24].

4. Current state: what exists and what does not

Evidence-based implementation map

Component	Current state	What that means
Dataset identity and schema checks	Implemented; saved manifests	Files, labels, counts, and fingerprints are recorded.
Training/validation split and preprocessing	Implemented; saved artifacts	Train-fit-only learning, reusable pipeline and feature order.
Centralized MLP training	Implemented; one recorded run	A basic model can learn using the prepared data.
Binary metrics and final evaluation	Implemented; saved report	Fixed threshold and separate official-test evaluation.
Ten-client non-IID partition	Implemented; three saved partitions	Alpha 0.5, ten clients, exact train-fit coverage, repeatable artifacts.
FL client and async server	Implemented; B0 pilot recorded	Thirty updates applied without a global-round barrier; exact arrival replay.
Demand, scheduler, update budgeting	Placeholders	No functioning adaptive controller.
Compression and transport byte accounting	Placeholders	Dense payload lengths recorded; full transport overhead and savings unverified.
Event monitoring and analysis	B0 logs implemented; broader modules planned	Pilot events exist; controller monitoring and main statistical outputs remain planned.
Network emulation	Planned; example YAML	No implemented testbed demonstrated by the inspected files.
Manuscript	Outline and contribution framing	Empirical contribution text is an intention, not a result.

The remaining unfinished modules raise `NotImplementedError`. The B0 server now validates and applies full model deltas, increments its version, and records staleness. A file name or configuration entry alone still does not prove a feature works.

Historical verification performed on 3 October 2026

The test suite passed: **54 tests in 7.36 seconds**, using the pre-existing Python environment at the old TA location. The migration record explains the count as 46 imported tests, two relocation tests, and six scaffold tests.

Those six FL-related tests contain placeholders or elementary checks. They do not prove asynchronous application, real byte accounting, fairness, policy isolation, or compression. The substantive evidence is strongest for Day 1.

Eight saved artifact references matched their SHA256 fingerprints: the checkpoint, saved run configuration, two manifests, preprocessing configuration, feature list, fitted pipeline, and split indices. This check did not retrain the model, rerun official-test evaluation, or rehash the raw CSVs.

Ruff reported one existing error: unused `staleness` in `src/async_server.py:9`. The system Python could not collect tests because pandas was missing; the existing TA environment successfully ran them. There is no project-local `.venv` in the inspected root.

Do not assign a completion percentage from "Day 1 of 30." The days have very different workloads, and the plan is an execution sequence rather than a measured schedule.

Sources: [S3–S8]; current checks recorded in [S12].

5. The existing results, explained honestly

What actually ran

One centralized MLP used seed 42, ten epochs, batches of 512, Adam with learning rate 0.001, dropout 0.1, and one CPU thread. It had 192 inputs, hidden layers of 128 and 64 units, one output, and **33,025 trainable parameters**.

The final epoch was saved; there was no hyperparameter search or early stopping in this run. The recorded **5.60 seconds** measures the training loop only, not complete preparation and evaluation. It is not a forecast of FL runtime.

Metric	Validation	Official test	How to read it
Macro-F1	0.935908	0.834634	Balances precision and recall within each class, then gives the two classes equal weight.
Attack recall	0.978885	0.981007	Fraction of actual attacks detected.
Average Precision	0.995007	0.980808	Summarizes attack precision/recall across score thresholds.
Accuracy	0.945507	0.843876	Fraction of all predictions correct.
AUROC	0.989537	0.973654	Measures how well scores rank attacks above normal examples.
Balanced accuracy	0.926630	0.828436	Gives normal recall and attack recall equal weight.

The code uses Average Precision for the reported AUPRC value, not trapezoidal integration. Attack predictions use a fixed score threshold of **0.5 or greater**.

The confusion matrix tells the practical story

Actual class	Predicted Normal	Predicted Attack
Normal	25,007 correct normal predictions	11,993 false alarms
Attack	861 missed attacks	44,471 detected attacks

The model detected **44,471 of 45,332 attacks: 98.10%**. But it also incorrectly flagged **11,993 of 37,000 normal records: 32.41%**. That false-positive rate is calculated from the saved matrix; it is not a separate saved metric field.

In plain English, the model catches most attacks but generates many false alarms at this threshold. A high attack recall or Average Precision does not establish that this model is suitable for operational deployment.

Validation Macro-F1 exceeds test Macro-F1 by about **10.13 percentage points**. The results alone do not identify the cause. Different class proportions are recorded, but distribution shift or overfitting would need separate evidence. Do not change settings to chase a better official-test score.

What this proves: the fixed data-to-model-to-metrics pipeline produced an auditable sanity result. **What it does not prove:** FL quality, communication efficiency, demand-controller usefulness, repeatability across multiple seeds, or deployment readiness.

Sources: [S4], [S5], [S7].

6. Understand the data and protect the experiment

Which data goes where

The project uses the official UNSW-NB15 training/testing CSV pair. It does not combine and resplit the four full-dataset CSV files.

Data group	Rows	Normal	Attack	Role
Official training	175,341	56,000	119,341	Source of train-fit and validation.
Train-fit	140,272	44,800	95,472	Learns preprocessing and model weights.
Validation	35,069	11,200	23,869	Held-out development evaluation.
Official test	82,332	37,000	45,332	Separate final evaluation.

Train-fit and validation are parts of official training; do not add all four rows to compute dataset size. The seed-42 split assigns 80% to train-fit and 20% to validation while preserving approximate class proportions.

```
Official training: 175,341 rows
    +-- Train-fit: 140,272 --> fit preprocessing --> train model
    |                       --> frozen ten-client partitions
    +-- Validation: 35,069 --> transform only --> evaluate

Official test: 82,332 -----> transform only --> final evaluation
```

The model must not see the answer

The target is `label`: 0 means Normal and 1 means Attack. Three columns are removed from the inputs: `id`, `attack_cat`, and `label`. A record identifier is not the intended traffic signal; attack category and label would reveal information about the answer.

The remaining **42 source features** contain 39 numeric fields and three categorical fields: `proto`, `service`, and `state`. Numeric fields are standardized. Categories become indicator columns through one-hot encoding. The fitted pipeline produces **192 ordered input features** in the recorded run.

Preprocessing learns only from train-fit. Validation and test only use the learned transformation. An unfamiliar category becomes all zeros within that categorical field rather than adding a new input column. Invalid labels, unexpected schema, missing values, and nonfinite numeric values are rejected.

What data leakage means

Data leakage means the process learns information it should not have when being evaluated. It can happen without explicitly training on test labels. For example, fitting a scaler on validation/test data lets held-out values influence the model's input preparation.

The initial audit reads both CSVs for identity, schema, values, counts, and hashes. After that, preparation and training open only the training CSV. The separate evaluation command opens official test after the checkpoint exists. Test results must not select demand weights, thresholds, architecture, or training settings.

For future FL, **only the 140,272 train-fit row positions go to clients**. Global validation and official test stay outside client training. Keep the existing split and provenance; do not repartition the entire official training set by accident.

Sources: [S3–S5], [S7], [S2, Day 2].

7. How the planned federated system works

A simple view of the whole system

Stage	What happens	Why it matters
Prepare	Transform data with the frozen pipeline; assign train-fit rows to clients.	All policies use comparable data.
Observe	Record the latest client workload, network condition, and contribution history.	Decisions use available evidence.
Decide	Convert state into participation and budget actions.	This is the proposed adaptive mechanism.
Dispatch	An eligible client receives a versioned global model.	The starting point of local training is known.
Train	The client performs the fixed local training procedure.	Compute does not become a hidden advantage.
Send	Form a model change, apply the chosen budget, serialize, and transmit.	Actual message size can be measured.
Apply	The server handles the arriving update and advances its version if accepted.	It does not wait for every client.

Record	Save versions, timing, state, action, bytes, and evaluation results.	The result can be explained and reproduced.

A **model delta** is the difference between the client's trained model and the model it received. Sending a delta communicates how the client changed the model; it does not send the client's raw training examples.

This research setup uses logical clients on a controlled testbed. It does not establish a fully decentralized data-collection or privacy system. Differential privacy and secure aggregation are outside the core scope.

Asynchronous does not mean "no coordination"

The server still controls eligibility, concurrency, model versions, and aggregation. "Asynchronous" means it can process an arriving update without a global wait for all clients.

Illustrative timeline: A and B both start from model version 20. A finishes first. Its update arrives when the server is at version 20, so its staleness is zero; the server applies it and becomes version 21. B arrives later while the server is at 21. B's staleness is one, and an accepted update advances the server to 22.

$$\text{staleness} = \text{server version before applying arrival} - \text{client starting version}$$

Staleness counts model-version changes, not seconds. Elapsed delay is a separate measurement. If the server waits for A and B together, the implementation does not meet the intended asynchronous behavior.

What is still open

The YAML names `weighted_delta` aggregation and `fixed` staleness weighting. Those names alone do not specify a complete mathematical or executable rule. The precise weights, acceptance/rejection behavior, concurrency semantics, and stop condition must be defined and tested before comparing policies.

The example concurrency limit is three clients. The technical plan treats final concurrency and several other values as pilot decisions. Present three as a scaffold value until the protocol freeze provides evidence otherwise.

Sources: [S1], [S2, Days 3–4], [S6], [S7].

8. Experimental infrastructure and RunPod ownership

Status and purpose: compute host, not the research method

Planned; no RunPod deployment is confirmed. RunPod is the proposed compute host and controlled test environment for the experiments. You are still deciding what resources to purchase. This guide records a design to test, not a purchased SKU, verified provider capability, or measured runtime/cost. Check the selected offering's current terms and capabilities before provisioning.

The research contribution remains demand-aware participation and update budgeting. RunPod would supply the environment in which you implement and compare those policies; renting more hardware is not the adaptive mechanism. In particular, the study does not dynamically allocate CPU, RAM, or GPU resources to favor B3.

Ten logical FL clients do not mean ten RunPod instances. A logical client is a separately identified participant with its own train-fit partition, model state, and contribution history. The documented design has ten clients. The proposed topology puts all ten on one host, alongside the asynchronous server. Ten separate rented instances would be a different topology with different cost and interference assumptions; that is not the current proposal.

Proposed one-host topology

```
One proposed RunPod compute host (SKU/resources TBD)
  |
  +-- Async server: versions, dispatch, arrivals, evaluation
  |
  +-- Controlled client/server network paths
  |     Network emulation and enforcement: planned, unverified
  |       +-- Logical client 01: partition + local model/state
  |       +-- ...
  |       +-- Logical client 10: partition + local model/state
  |     Maximum active training jobs: pilot/freeze decision
  |
  +-- Persistent experiment storage (type/mount/size TBD)
        Data + manifests + configs + checkpoints + event logs
        Results + environment receipt + experiment ledger
```

Planned isolation: separate client processes or containers, with enforceable compute limits and network paths. The choice is open. Linux/Docker/cgroup controls are part of the proposed testbed, not verified capabilities of a selected RunPod offering. Having ten client identities does not imply ten simultaneously training jobs. The concurrency value of three in Section 7 is only a scaffold value.

Each client would receive only its assigned train-fit rows. Validation and official test remain outside client training. Co-locating the data on one host simulates FL behavior; it does not establish independently administered data owners or a formal privacy boundary.

The server must process arrivals without a global barrier. Network emulation must affect the actual client/server message path, rather than merely attaching a delay label to a log. Isolation, routing, and shaping feasibility need a smoke test before this topology can be described as working.

Shared CPU, memory, GPU, disk, and evaluation work can create contention. This can change training time and arrival order even with identical nominal allocations. Record utilization and timing during pilots, reserve server/evaluation headroom, and control unrelated workloads. One-host results would support a controlled emulation claim, not a claim about ten independent physical sites or Internet performance.

Resource contract: exact design values and placeholders

No hardware quantities below are measured or frozen. Complete the TBD entries from a sizing pilot, then record one resource contract shared across B0–B3. Keep allocated limits separate from observed utilization.

Component	Value/status today	What must be recorded before main runs
Server	One async server, planned; CPU/RAM TBD	CPU quota/affinity, RAM limit, evaluation placement and headroom.
Clients	Ten logical clients, documented design	Stable IDs and partition links; per-client CPU/RAM limits and thread counts TBD.
GPU	Requirement, count and VRAM TBD	CPU-only or GPU execution; sharing/isolation, precision and allocation rules.
Concurrency	Three is a scaffold example; final value open	Maximum active training jobs, queue semantics and any evaluation overlap.
Storage	Persistent storage planned; type/size TBD	Mount, usable capacity, retention, backup/export location and survival checks.
Host	One host proposed; SKU/CPU/RAM/GPU TBD	Actual usable resources, image/environment identity and capability receipt.

Do not equate ten clients with ten GPUs or multiply active-job resources by ten without deciding what remains resident. Idle clients may still retain models, partitions, or other state. Host sizing must include the server, evaluation, active and idle clients, dataset/preprocessing residency, operating environment, and safety headroom. How much each component needs is **TBD until measured**.

Fixed-resource fairness across B0–B3

Planned experimental control: keep the host class, server allocation, per-client limits, GPU assignment/sharing rules, software/thread settings, maximum concurrency, and storage configuration matched within each paired condition. Use the same model/training contract, partition, seed, aggregation rule, evaluation cadence, network profile and client-to-profile mapping described in Sections 7 and 10.

B1/B3 may change who becomes eligible and when; B2/B3 may change update size. Actual occupancy, contribution counts, communicated bytes, and elapsed time may therefore differ. Fixed capacity does not mean forcing equal realized utilization or equal client contributions. It means no policy receives extra hardware or a different concurrency ceiling as an unreported advantage.

Proposed run scheduling: execute one policy run at a time on the host, with fresh per-run state and output paths. Paired runs share the frozen configuration, not residual models, queues or compression state from a previous policy. Freeze the run order and reset procedure before main runs; monitor host load and record deviations. Running B0–B3 together could introduce cross-policy contention and needs a separate validated protocol before use.

Purchase checklist and open infrastructure decisions

Use a small sizing pilot to decide what to buy; this guide does not recommend a current SKU or price. A pilot provision is itself a cost decision, so record its budget and stop limit before starting. Availability, permissions, and persistence behavior must be checked for the actual offering.

Open decision	Evidence needed to close it
SKU and CPU/RAM/GPU	Peak memory, CPU load, local training and evaluation time; whether GPU execution is needed and beneficial for this workload.
Client isolation	Process/container choice; verified CPU/RAM enforcement, thread behavior, GPU sharing and separate network paths.
Concurrency	Pilot throughput, contention and server responsiveness at candidate limits; select and freeze one ceiling.
`tc/netem` shaping	Required permissions/tools; interface/namespace layout; bandwidth mechanism, delay/loss settings, directions, and measured behavior.
Persistent storage	Required capacity, actual mount and lifecycle behavior, backup destination, retention and restore test.
Recovery	Restart-from-scratch versus resume; checkpoint completeness, duplicate-update handling and failure ledger policy.
Runtime and cost	Measured pilot durations, full run count, retry allowance, storage/transfer charges and spending/stop limits.

Planned shaping gate: verify the selected environment can support the required controls before relying on it. Specify upload/download directions, how a target RTT maps to applied delays, bandwidth enforcement, client-to-profile assignments and any time-varying trace. Validate achieved latency/throughput through the same paths used by experiment messages. Do not claim example YAML values are measured link behavior. Exact commands, interfaces, privileges and shaping rules are **TBD**; if enforcement fails, resolve the topology or implementation before claiming network-controlled results.

Planned persistence/recovery gate: inventory everything required to reproduce a run, test artifact survival across the intended host lifecycle, and restore a disposable test run from the chosen storage/backup. Storage called persistent is not evidence that every required file survives every lifecycle action. Exact RunPod lifecycle guarantees remain unverified here. A model checkpoint alone may not capture server versions, client state, queues, optimizer/compression state, random state and logs needed for faithful resume.

Proposed conservative recovery default: until resume is implemented and tested, mark interrupted runs failed and restart into a fresh run directory from the frozen inputs. Preserve the failed run and its incurred runtime/cost. Never silently stitch partial runs together or omit failures from the ledger. Resume remains an open decision requiring a separate validation gate.

Runtime and cost worksheet — inputs TBD

The guide's planned matrix is four policies × three seeds × two profiles = **24 main runs**, before pilots, calibration, retries or optional conditions (Section 10). It is not evidence that all runs take equal time.

Record pilot hours, calibration hours, expected main-run hours, retry allowance, setup/idle hours, storage capacity and billed duration, and any applicable transfer or other charges. Record the price source/date and billing units only after checking the chosen offering.

```
Planned compute hours = setup/idle + pilots + calibration
                      + sum(expected hours for each main run)
                      + retry allowance
Estimated total cost  = compute hours x applicable hourly rate
                      + storage + transfer + other applicable fees
All time, rate, capacity and fee inputs: TBD
```

If rates differ across phases, calculate each phase separately. Define a spending ceiling and shutdown procedure before provisioning. A cheaper hourly rate need not mean a cheaper completed experiment; use measured workload duration and verified capabilities to justify the final choice.

Boot-to-experiment reproduction flow — planned, not executable yet

The following is an intended operational sequence. The Day 1 commands in Section 15 remain historical/current documentation, and `src/b0.py` now provides a local CPU pilot launcher. Infrastructure bootstrap, shaping scripts, full B0–B3 launch and operational recovery remain **TBD**.

1. **Freeze the run specification.** Select the policy, seed, partition, network condition, stopping/evaluation rules and resource contract. Allocate a unique run ID and fresh output directory; record budget/stop limits. Do not launch main runs while these remain open.
2. **Provision and identify the host.** Select the pilot-supported offering and image; record actual CPU/RAM/GPU, software versions and available permissions. Verify required isolation/network capabilities rather than assuming them from a SKU name. Provision/bootstrap commands: **TBD**.
3. **Attach and verify storage.** Confirm the intended persistent mount and free capacity; check write access and the backup/export destination. Restore data, manifests, preprocessing artifacts and source/configuration identities. A Git clone alone does not supply ignored datasets or artifacts. Attach/restore commands: **TBD**.
4. **Establish the environment.** Follow the existing installation contract; record exact installed versions and image identity. Check artifact fingerprints and use the existing test/lint workflow with the actual environment path. The historical lint issue is not a newly resolved result. Environment/bootstrap and complete FL checks: **TBD**.
5. **Validate isolation and transport.** Start a disposable server/client smoke test, verify resource enforcement and actual message routing, then measure shaped latency/throughput and byte accounting. Shaping and smoke-test commands: **TBD**.
6. **Run the sizing pilot and freeze.** Measure memory, utilization, contention, server responsiveness, training/transfer/serialization time and disk growth. Resolve resource, concurrency and shaping choices; test persistence and failure handling. Pilot launcher and acceptance thresholds: **TBD**. Keep pilot results distinct from main results.
7. **Launch the experiment.** Start the asynchronous server and ten logical clients, enforcing the frozen active-job ceiling. Verify versioned arrivals and absence of a global barrier. Run B0–B3 sequentially under paired conditions and fresh state. FL launch/orchestration commands: **TBD**.
8. **Record completion or failure.** Save status, stop reason, versions, events, bytes, timing, resource/network receipt, checkpoints and validation observations. Mark unreached targets and interruptions explicitly. Perform official-test evaluation only under the frozen evaluation protocol, never as repeated tuning.
9. **Archive, verify and shut down.** Verify output fingerprints and backup/export completion; retain the run ledger, source/config identities and failed-run evidence. Stop billable compute according to the verified lifecycle procedure; record final runtime and charges. Archive/shutdown commands: **TBD**.

Acceptance before main runs: one end-to-end smoke run with enforceable limits and measured shaping, one sizing receipt supporting the purchase, one tested restore/failure procedure, and a frozen protocol/resource contract. These are proposed gates; none is reported complete here. Exact numerical tolerances remain open until the pilot protocol is defined.

Examiner-style infrastructure ownership questions

Suggested answers below defend the proposal; they are not deployment evidence.

What does RunPod contribute to the TA? A proposed compute host and repeatable test environment. The evaluated contribution is the communication policy, not the hosting provider.

Why do ten clients not require ten rented instances? Clients are logical participants with separate partitions/state. The proposed experiment hosts ten on one machine; concurrency limits how many train at once.

Why one host, and what does it fail to establish? It is a proposed way to control and operate the testbed with fewer separate hosts. Feasibility and cost still need a pilot. Shared-host emulation cannot establish independent-site behavior, real Internet performance, or formal privacy.

Which CPU/RAM/GPU resources did you actually use? None are confirmed for RunPod. The table distinguishes documented client count and scaffold concurrency from hardware quantities awaiting a pilot.

How will you prove the slow client is really slow? Verify shaping on the experiment's message path and record measured latency/throughput. A configured value or a sleep in unrelated code is not sufficient evidence.

How do you stop B3 winning because it gets more compute? Freeze the same allocations, active-job ceiling, training contract and matched conditions for all policies; record resource utilization and deviations. Only the intended participation/update-budget controls vary.

Could the server or GPU become the bottleneck? Yes, that is a pilot question. Measure contention and server/evaluation responsiveness, then freeze supported limits before main comparisons.

What happens if the host disappears mid-run? Recovery is not validated yet. The proposed default is a recorded failed run and fresh restart; faithful resume requires complete state and separate tests. Persistence and backup must be demonstrated.

Can someone reproduce the experiment from Git alone? No. They also need fingerprinted datasets/artifacts, partitions, exact environment and frozen configs, plus the eventual operational scripts and logs.

Why buy that particular configuration? No SKU is selected here. The defensible answer will link a sizing pilot, capability checks, runtime estimate and spending ceiling to the chosen offering.

What remains yours to decide and verify? SKU/resources, isolation, concurrency, shaping, storage, recovery, run budgets and cost controls. Owning the experiment means explaining each choice and showing its evidence, not reciting a provider name.

Evidence boundary: client design, scaffold concurrency and shared experimental controls are described in Sections 7 and 10 and the historical sources [S1], [S2], [S6]. The one-host RunPod topology, sizing gates, resource placeholders, recovery default and operational flow above are additions proposed on 5 October 2026. No provider-specific capability, price, deployed resource or completed RunPod experiment is asserted.

9. Demand and the two controls

What the controller observes

"Demand" is a constructed decision signal. It is not directly measured like a file size. The proposed estimator combines observable client state into scores used for action selection.

State field	Plain-English meaning	Important question
`num_samples`	Number of local training examples.	How should data quantity influence opportunity or budget?
`prev_local_train_sec`	Time taken by the previous local training contribution.	Is training likely to delay the next update?
`rtt_ms`	Network round-trip delay.	How expensive is communication latency?
`throughput_mbps`	Available/measured transfer rate.	How much update information can move efficiently?
`staleness`	Age of an update in server versions.	Is the client regularly contributing outdated changes?
`age_since_last_update`	Time or events since the last contribution.	Has the client been ignored too long?

The exact units and availability time of each state field need freezing. For example, the duration of an unfinished training job cannot be used as if it were already known. Start-up defaults are also needed when a client has no previous contribution.

One module, two scores

```
Observable client state
          |
Calibration-based normalization
          |
Transparent demand scoring
          +-- d_part   --> participation action
          +-- d_budget --> update-size action
```

Normalization puts unlike quantities on usable scales. The configuration proposes min-max normalization from calibration runs. The scoring formula, feature directions, weights, thresholds, and treatment of unusual values are not implemented. Do not invent final coefficients and describe them as established research decisions.

Two outputs avoid pretending that “should participate sooner” always means “should send a larger update.” A neglected client with a slow link can create competing needs. The controller must make those trade-offs explicit.

Control 1: participation

The initial action grid uses cooldowns of three, one, or zero server-update events for low, medium, or high participation levels. A cooldown determines when the client becomes eligible again; it is not the number of clients in a synchronous round.

Being eligible does not necessarily mean being dispatched immediately if concurrency is full. Tie-breaking and the interaction between priority and cooldown must be defined. A maximum inactivity rule prevents a client from being ignored indefinitely. The scaffold uses 20 events; the technical plan still requires its final value to be frozen.

Control 2: update budget

The proposed budgets retain **25%, 50%, or 100% of model-delta entries**. Magnitude top-k keeps the entries with the largest absolute changes. It must also transmit enough position information to reconstruct the update.

Nine action pairs are possible: three participation levels times three budgets. The budgets are not guaranteed percentages of actual network bytes. If error feedback is used to carry omitted changes forward, its rule must be identical across comparable policies using it and frozen before main runs.

Sources: [S1, Sections 13–17], [S2, Days 8–14], [S6–S7].

10. B0–B3: the experiment that isolates the idea

The four policies

Policy	Participation	Update budget	What it tests
B0: Async-Uniform	Fixed/uniform	Full, 100%	The main baseline on the same asynchronous system.
B1: Demand-Participation	Demand-aware	Full, 100%	The effect of changing participation alone.
B2: Demand-Budget	Fixed/uniform	Demand-aware	The effect of changing update size alone.
B3: Joint Demand-Aware	Demand-aware	Demand-aware	The combined method.

An **ablation** removes part of a method to see what that part contributes. B1 and B2 are essential: without them, a B3 win would not tell you whether the joint design helps beyond a simpler single control.

The centralized Day 1 model is a pipeline sanity check. It is not B0. Likewise, synchronous FedAvg may provide later context, but it is not the main causal comparator.

What must stay matched

Within each paired seed and condition, hold fixed the client partition, preprocessing, model initialization, architecture, local epochs, batch size, optimizer, learning rate, precision, compute limits, aggregation and staleness rules, maximum concurrency, network profile/trace, and evaluation protocol.

The proposed host resource contract and contention controls are in Section 8.

“Uniform participation” means a common scheduling rule. In an asynchronous system, that does not automatically produce identical contribution counts: faster clients may finish more frequently. Count contributions and inactivity rather than assuming the baseline is perfectly balanced.

Data differences versus system differences

Non-IID data means clients do not have the same statistical mix of examples. The frozen Day 2 partitions use Dirichlet label skew with ten clients and main alpha 0.5. Smaller alpha, such as 0.1, generally gives more uneven label mixtures. Alpha 10 is a planned near-IID control, not a guarantee of identical clients.

System/network heterogeneity means clients differ in speed or connectivity. Keep the client data assignment fixed while changing the network condition, so the sources of variation remain separable.

Example profile in YAML	RTT	Bandwidth
Normal	20 ms	100 Mbps
Heterogeneous: fast	10 ms	100 Mbps
Heterogeneous: medium	50 ms	30 Mbps
Heterogeneous: slow	150 ms	5 Mbps

These values are scaffold examples awaiting the pilot/protocol freeze. The client-to-class assignment and repeatable network behavior also need specification.

The current main configuration lists seeds **42, 123, and 2026**. Four policies times three seeds means **12 runs per profile**. Covering both listed profiles means **24 main-condition runs**, excluding calibration, pilots, retries, and optional alpha extensions. This is planned workload, not completed output.

Sources: [S1], [S2, Days 11–21], [S6].

11. Measurement: bytes, time, quality, and fairness

Count what is actually sent

For each contribution, count serialized upload and download messages, including relevant indices, headers, and metadata. Define how any transport envelope or additional traffic is included. A payload-length measurement should not be mislabeled as every physical-network byte unless that boundary is actually measured.

Illustrative arithmetic, not a project result: a dense update of 100 float32 values has 400 value bytes. Keeping 25 values uses 100 value bytes, but 25 int32 indices add another 100 bytes. Even before metadata, that is 200 bytes, so a 25% retained-value budget has produced only a 50% upload reduction in this example.

If a 400-byte download is still required, total traffic changes from 800 to 600 bytes: only 25% less. This is why upload compression alone is not enough to establish total communication savings.

Know which endpoint answers which question

Endpoint	Meaning
Total bytes	All counted upload plus download bytes during the run.
Bytes-to-target	Cumulative bytes at the first validation observation reaching the frozen Macro-F1 target.
Time-to-target	Elapsed time at that same first qualifying observation.
Accepted updates to target	Number of applied client updates needed to reach it.
Serialization/compression time	Time preparing the message representation.
Transfer time	Time spent moving the message under the defined transport.
Local training time	Time spent updating the local model.
Staleness	Server-version age of an arriving update.

Do not add overlapping concurrent durations and call the result wall-clock time. Define start/end timestamps and timing boundaries. Periodic evaluation also limits target-crossing precision: the first observed crossing can be later than the true crossing.

A run that never reaches the target must be labeled **not reached** or **censored**, with its stopping limit recorded. It must not become zero cost or quietly disappear from the comparison.

Quality and fairness safeguards

Report Macro-F1, attack recall, and Average Precision as the primary model metrics. Accuracy, AUROC, balanced accuracy, and the confusion matrix provide supporting context. A low communication bill is not a success if important attacks are missed or normal traffic generates excessive false alarms.

For fairness, record each client's participation share, longest inactivity, and staleness distribution. Mean staleness alone can hide a badly affected minority; include median and high-percentile values. The optional Jain index summarizes how evenly contributions are spread, but does not show whether rare or useful examples are represented.

The policy needs an explicit inactivity safeguard because excluding a slow client may also exclude distinctive data. Fairness is therefore relevant to both opportunity and model quality.

Sources: [S1, Sections 16 and 22], [S2, Days 5, 12, and 23]. The byte example above is explanatory arithmetic.

12. Analysis and claims you can defend

Compare like with like

For each seed, compare B3 with B0 under the same data and system conditions. Compute a difference for that pair, then summarize the differences across seeds. Also compare B1/B0, B2/B0, and B3 against the individual controls to explain the mechanism.

The plan calls for means, medians, standard deviations, differences, confidence intervals, and meaningful effect sizes. It proposes paired bootstrap analysis: repeatedly resample whole seed pairs and recalculate the summary. Save the resampling seed and interval method.

The independent repetition is the run/seed, not each update event. Thousands of updates from one run do not become thousands of independent experiments. Three seeds are the minimum plan and give limited uncertainty information; five seeds are the publication target.

Understand non-inferiority

The question is whether the quality loss is small enough, not simply whether a conventional test finds a difference. The planned condition is:

$$\text{lower 95\% confidence bound of } (\text{Macro-F1}_{B3} - \text{Macro-F1}_{B0}) > -0.01$$

Example: an interval from −0.006 to 0.003 meets that criterion. An interval from −0.020 to 0.004 does not, even if the average difference is small. The second outcome can be inconclusive rather than proof of harm. These are teaching examples, not measured intervals.

Explain trade-offs rather than selecting one flattering number

A Pareto comparison asks whether one option can improve a result without worsening another. Plot communication, time, and quality together. A method that is cheapest but slowest or least accurate may be useful in some settings, but it is not automatically the best method overall.

The plan includes failure analysis: inspect at least 20 informative cases when available and trace them from client state to demand to action to outcome. Candidate categories include starvation, stale updates, budgets that discard useful information, unnecessary large updates, metadata overhead, and misleading demand scores. If fewer cases exist, report the available count and why.

Limits that belong in the thesis

- The non-IID distribution is simulated label skew; it is not a measured real-world client distribution.
- Containers on one host and network emulation do not prove performance on real edge devices.
- One dataset limits generalization; ROAD is a separate later replication.
- Fixed preprocessing across clients is a controlled experiment choice, not proof of fully private distributed preprocessing.
- Update sharing alone is not a formal privacy guarantee; privacy defenses are outside scope.
- Sparse updates may interact with client imbalance and staleness; logs are needed to explain the effect.
- Small seed counts limit the strength of statistical conclusions.

If many secondary statistical tests are conducted, the research README calls for Holm correction. Keep the primary question explicit so exploratory observations do not quietly become the headline hypothesis.

Sources: [S1, Sections 24 and 33], [S2, Days 24–26]. Additional explanations clarify the limits of the planned setup.

13. Future plan: a practical route to completion

The repository provides a 30-day plan. Treat these as ordered work packages with acceptance gates, not a promise that each package will take exactly one calendar day. Days 1–3 are completed through the B0 pilot; Days 4–30 are planned.

Week 1: make the baseline and measurement trustworthy

Day	Work	Evidence required before moving on
1	Data and centralized sanity baseline	Saved pipeline, split, model, metrics, provenance, substantive tests. Completed.
2	Ten-client Dirichlet partition	Exact coverage of train-fit, no overlap, no held-out rows, repeatable manifest. Completed for three seeds.
3	B0 asynchronous training	Updates applied as they arrive; fixed local training and full budget. Completed CPU pilot with replay.
4	Versioning and staleness	Controlled fast/slow-client test and correct version history.
5	Byte accounting	Measured serialized messages; upload/download and timing separated.

Day	Work	Evidence required before moving on
6	25/50/100% update budgets	Correct reconstruction, shape/type preservation, actual byte counts.
7	Backbone freeze	Versioned configuration and passing evidence for Days 1–6.

Do not build the adaptive policy on top of an unverified B0. Otherwise, a policy change may hide a server or measurement bug.

Week 2: build the controller and freeze the protocol

Day	Work	Evidence required
8	State monitoring	Defined units, observation times, and state snapshots.
9	Calibration	Saved normalization values from permitted data/runs.
10	Demand estimator	Deterministic state-to-score-to-action replay.
11	B1 and B2	Only the intended single control changes.
12	B3 and fairness safeguard	Joint action works and inactivity rule is enforced.
13	Four-policy pilot	Complete logs and a documented issue/fix record.
14	Protocol freeze	Weights, thresholds, target, stopping rules, profiles, and analysis decisions fixed.

Calibration and pilot runs are for making the protocol workable. Keep their artifacts separate from the main evaluation results.

Week 3: collect the main evidence

Days 15–18 run B0–B3 for at least three seeds at alpha 0.5. Days 19–21 repeat under controlled network heterogeneity. Every paired comparison must reuse the matching partition, model initialization, training settings, and network condition.

Save complete runs, failed runs, and target-not-reached runs. Record why a run failed or was repeated. Do not retain only the most favorable seed.

Week 4: explain, reproduce, and finish

Day	Work	Output
22	Summarize raw logs	One row per run, with reconciled totals.
23	Calculate target endpoints	Bytes/time-to-target and reachability results.
24	Paired statistics	Differences, uncertainty, and non-inferiority analysis.
25	Ablation and trade-offs	Explanation of individual and joint effects.
26	Failure analysis	Traceable cases, mechanisms, and limitations.
27–28	Write the paper	Methods, setup, results, discussion, threats to validity.
29	Reproduce an output	At least one main table/figure regenerated in a fresh environment.
30	Final research freeze	Code/config identity, manifests, logs, figures, manuscript, and run ledger.

Optional work comes after the core gate

The article-strengthening plan includes five seeds, alpha 0.1 and 10 robustness, demand-feature ablation, and ROAD replication or a small physical cluster. ROAD should have its own validated preparation and experiment; do not merge it into UNSW-NB15 or assume the same learned weights transfer.

Adaptive local epochs, adaptive batch size, adaptive CPU/memory, reinforcement learning, a new compression algorithm, privacy defenses, poisoning defenses, multiple model architectures, live edge deployment, and a production dashboard are outside the first-month core scope.

Completion gate: B0–B3, repeated paired runs, actual bytes, target endpoints, quality and fairness metrics, ablations, uncertainty, failure analysis, and at least one reproducible main output. The guide itself does not complete these research tasks.

Sources: [S1, Sections 21 and 34–38], [S2].

14. Open decisions and immediate priorities

Decisions that still need an explicit record

Open item	Why it matters	When to resolve it
Complete aggregation/staleness equation	Policy results depend on how arriving updates change the model.	Before backbone freeze.
Concurrency, eligibility, dispatch, tie-breaking	Otherwise “participation” is ambiguous.	Backbone/pilot freeze.
Stop conditions and run budgets	Shorter or incomplete runs can look artificially cheap.	Before main runs.
Demand formula, directions, weights, thresholds	This defines the proposed method.	Calibration and Day 14 freeze.
Missing/start-up/out-of-range state handling	A controller must work before prior measurements exist.	State/calibration stage.
Top-k ties, rounding, payload, reconstruction	Needed for deterministic budgets and valid byte comparisons.	Days 5–6.
Error feedback, if used	Retained residuals change later updates.	Before policy comparison.
Fairness rule and inactivity value	Prevents an adaptive policy from ignoring clients.	Pilot/protocol freeze.
Network assignments and enforcement	Example YAML is not a measured, repeatable network.	Testbed/pilot freeze.
Target, evaluation interval, not-reached handling	Defines bytes/time-to-target and fair statistical analysis.	Before main runs.
Primary endpoint and quality guardrails	Prevents choosing whichever result looks best afterward.	Analysis-plan freeze.

RunPod-specific purchase, isolation, concurrency, shaping, persistence, recovery, runtime and cost decisions are collected in Section 8. They remain open; no deployment is confirmed.

Some entries are explicit open decisions in the technical plan; others are implementation details that the inspected scaffold does not resolve. This list identifies gaps, not new approved features.

Your immediate next work package

Verify Day 4 versioning and staleness under controlled fast/slow conditions. Use the saved B0 event trace and extend its version-transition tests. The following partition gates have already passed for seeds 42, 123, and 2026:

1. The union of client rows is exactly the train-fit set.
2. No two clients share a training row.
3. Client row counts total exactly 140,272.
4. No validation or test row is assigned to training clients.
5. The same inputs and seed reproduce the same assignment.
6. The manifest records source linkage, seed, algorithm, and ordering rules.

Day 2 uses per-class `PCG64(SeedSequence([partition_seed, class_label]))`, shuffle-before-draw, and floor cumulative cut points. Empty clients are retained without resampling or rebalancing; none occur in the three frozen partitions. See `data/partitions/README.md` for counts, hashes, replay rules, and checks. Day 3's no-barrier test, real event log, and replay now verify immediate apply with fixed training and full budget. Its pilot aggregation and concurrency choices are recorded, with main-experiment freeze still pending.

Documentation differences to keep straight

Day 1 uses dropout 0.1, batch 512, and ten centralized epochs. The future FL configuration uses dropout 0.2, batch 64, and one local epoch. These are separate configurations, not evidence that the archived baseline used the FL values. The future B0–B3 model/training contract must be frozen and shared across policies.

Older documents mention a nested repository location. The root inspected for this guide is `/home/fadil/DSIC-2907`. Archived paths preserve historical provenance; their relocation map resolves local copies. Do not "fix" old manifest text casually, because changing it changes its fingerprint.

Sources: [S2], [S3], [S6–S8].

15. Repository ownership and safe operation

Where to find things

Location	What you own there
`README.md`	Overall research design, questions, scope, experiments, and completion criteria.
`teknis-lab/README.md`	Detailed 30-day implementation/evidence plan.
`paper/day1/`	Completed Day 1 instructions, tasks, and migration history.
`paper/docs/Jurnal/`	Explanation and interpretation of the completed baseline.
`paper/manuscript.md`	Article outline and contribution framing.
`configs/day1_*.yaml`	Settings for the archived centralized baseline.
Other files in `configs/`	Proposed FL, policy, demand, and network settings.
`src/preprocess.py`	Data audit, split, transformation, artifact verification, and relocation.

Location	What you own there
`src/centralized.py`	Existing model, training, checkpoint, and separate evaluation.
`src/metrics.py`	Binary metric and threshold definitions.
`src/partition.py`	Deterministic train-fit partitioning and artifact generation.
`src/client.py`, `src/async_server.py`, `src/b0.py`	Fixed local training, weighted immediate apply, CPU pilot launcher and arrival replay.
Other files in `src/`	Planned FL/controller/measurement modules, currently placeholders.
`tests/`	Day 1 protection, 21 partition checks, 19 Day 3 checks, and five limited remaining scaffold checks.
`data/manifests/`	Dataset/artifact identity and relocation records.
`data/partitions/`	Three frozen ten-client partitions, class-count CSVs, and provenance manifests.
`data/processed/`	Fitted pipeline, split indices, features, and saved preparation settings.
`results/raw/centralized_seed42/`	Model checkpoint, training receipt, and configuration.
`results/raw/b0_day3_seed42/`	Pilot snapshot, initial/final/periodic models, serialized messages, events, validation and version history.
`results/processed/`	Saved centralized official-test result plus B0 pilot summary and replay receipt.

Python and PyTorch support training; NumPy, pandas, scikit-learn, and joblib support data and artifacts. SciPy and matplotlib are declared for statistics/plots. Linux, Docker, cgroup v2, and network controls belong to the planned testbed. The archived Node package files record old workspace tooling; they are not a research application stack.

Understand the evidence chain

```
Saved result
   --> checkpoint fingerprint
   --> preprocessing-manifest fingerprint
   --> dataset-manifest fingerprint
   --> source CSV fingerprints

Preprocessing manifest also identifies:
   fitted pipeline + split indices + feature order + configuration
```

A SHA256 fingerprint helps detect changes or accidental mixing of files. It is not independent proof that the publisher authenticated the dataset or that the entire experiment was correct.

The original run records `git: null`; it predates the current repository linkage. Do not retroactively claim that the current commit was the commit that generated that run. Preserve both the archived provenance and the migration history.

Inspect before you execute

To read results without training or evaluating again:

```
cd /home/fadil/DSIC-2907
rtk proxy python3 -m json.tool \
  results/processed/centralized_seed42_test.json
rtk proxy python3 -m json.tool data/manifests/preprocessing.json
rtk git status --short
```

The normal future verification commands, after establishing a project environment, are:

```
rtk proxy .venv/bin/python -m pytest -q
rtk proxy .venv/bin/python -m ruff check src tests
rtk proxy .venv/bin/python -m ruff format --check src tests
rtk proxy .venv/bin/python -m build
```

These are workflow commands, not a claim that all passed in this session. The current guide's actual checks are in Section 17. The existing unused-variable lint failure remains.

For a clean independent setup, follow the Day 1 installation instructions with the **current** repository root, using Python 3.11 or newer and an isolated environment. CPU PyTorch is sufficient for the baseline. Declared version ranges are not an exact environment lock; the saved report contains the original run's recorded versions. No dependencies were installed while making this guide.

The execution sequence is `freeze` dataset identity, `prepare`, `train`, then separate `evaluate`. Existing outputs are deliberately protected from overwrite. Independent reproduction needs fresh output paths and a copied configuration; it must not become repeated official-test tuning. There is no implemented FL run command to give you yet.

What to back up

Keep source/configuration, small manifests, raw datasets, fitted pipeline, split indices, checkpoint, run receipts, and results together with their fingerprints. Large artifacts and raw data are Git-ignored; a clone alone does not restore the whole experiment.

For the proposed RunPod setup, use the persistence/restore gate in Section 8; storage and lifecycle behavior have not been validated.

For future runs, also retain raw event logs, network profiles, partition manifests, analysis outputs, and failure reasons. Preserve an experiment ledger with run ID, policy, seed, status, failure reason, source identity, and config identity. Reproduction should regenerate evidence from saved inputs, not rely on manually edited tables.

Sources: [S3], [S6–S8], [S11].

16. Taking ownership: explain it, question it, defend it

A one-minute explanation you can adapt

> My TA studies communication efficiency in asynchronous federated intrusion detection. Ten logical clients train the same type of model on different mixtures of normal and attack records. The server accepts updates without waiting for all clients. I want to test whether client workload and current conditions can guide two decisions: how soon a client participates and how much of its model update it sends. I compare four policies on the same backbone so I can separate each decision's effect. I measure actual communicated bytes and time to a fixed quality target, while protecting detection quality and participation. So far, I have completed preprocessing and one centralized sanity baseline. The federated and adaptive components are still to be implemented and evaluated.

Questions an examiner or supervisor may ask

What is your actual contribution? A controlled evaluation of joint state-based participation and update-budget control, with an explicit demand interface, component ablations, real serialized-byte measurements, and a quality tolerance.

Why asynchronous FL? It is the common backbone of the approved design. It allows immediate handling of arrivals and creates the staleness/speed conditions the policy must manage. The main study does not test whether asynchronous beats synchronous FL.

Why not compare only B0 and B3? Because B1 and B2 reveal whether each control helps and whether their combination provides additional value.

What does demand mean? Two constructed scores from observed state, one for participation and one for budget. The exact formula still needs calibration, documentation, and freezing.

Why keep computation fixed? So communication-policy effects are not mixed with different amounts of model training or different hardware allocation.

Does a 25% budget save 75% communication? Not necessarily. Values, indices, metadata, and downloads must all be counted according to the defined measurement boundary.

Why is attack recall high but Macro-F1 lower? The saved model catches many attacks but also generates many false positives. Macro-F1 includes how well the normal class is handled.

How do you prevent test leakage? Fit preprocessing on train-fit only, keep validation separate, reserve official test for frozen evaluation, and never use test results to choose controller or model settings.

Is your non-IID setting realistic? It is a controlled simulation of unequal label mixtures, useful for comparison. It is not evidence of a real organization's client distributions.

What if B3 fails? Report the failed hypothesis, identify when and why it fails, and explain the trade-off. A valid method does not require a favorable outcome.

Can you claim privacy? The intended FL flow avoids routine raw-example sharing during local training, but no formal privacy guarantee or secure aggregation is implemented in the core scope.

What has been completed? The audited data/preprocessing pipeline, one fixed centralized model, metrics, saved provenance, three verified ten-client partitions, and a 30-update B0 asynchronous pilot with exact model replay. The main FL comparison remains planned.

Your ownership checklist

Before saying you fully understand the TA, be able to:

1. Draw the data split and explain why clients receive only train-fit rows.
2. Explain every number in the saved confusion matrix.
3. Distinguish the centralized baseline from B0.
4. Walk through two asynchronous arrivals and calculate staleness.
5. Explain all six state fields and both demand outputs.
6. Name what changes and what stays fixed in B0–B3.
7. Explain why a smaller tensor does not necessarily mean the same percentage of byte savings.
8. Define target-reaching, non-inferiority, and a paired seed comparison.
9. Find the source artifact behind a result and explain its fingerprint chain.
10. Name the next work package, its gate, and the decisions still open.
11. Draw the proposed RunPod topology and distinguish logical clients, active jobs, and rented instances.

12. Explain the resource table, shaping/restore gates and sizing evidence needed before buying a host (Section 8).

For each future result, ask: **What did we change? What stayed the same? What was measured? Where is the raw evidence? What conclusion is justified?** That habit matters more than memorizing terminology.

Based on the project design and verified status in [S1–S12]. The speaking script is suggested wording, not a claim of completed FL work.

17. Source map and verification record

Where the statements come from

All paths below are relative to `/home/fadil/DSIC-2907`. Source labels throughout the guide point here. This is an internal evidence map, not a substitute for the thesis bibliography.

[S1] Research design: `README.md`. Defines the objective, scope, questions, hypotheses, policies, endpoints, validity limits, and research/article completion gates.

[S2] Execution protocol: `teknis-lab/README.md`. Gives the Day 1–30 work sequence, partition invariants, open pilot values, protocol freeze, analysis, and final artifact requirements. Existing local edits were included in the inspected snapshot.

[S3] Day 1 documentation: `paper/day1/README.md`, `paper/day1/tasks/plan.md`, and `paper/day1/tasks/todo.md`. Describe the completed implementation and its historical verification.

[S4] Baseline explanation: `paper/docs/Jurnal/Day-1-Preprocessing-dan-Baseline-Terpusat.md`. Explains preparation, results, false alarms, provenance, and limitations.

[S5] Recorded numerical result: `results/processed/centralized_seed42_test.json`. Authoritative saved values for the metrics, matrix, counts, model settings, training duration, package versions, and linked artifact identities used in this guide.

[S6] Configuration: files under `configs/`: the two `day1_*.yaml` files plus `model.yaml`, `async_fl.yaml`, `demand.yaml`, `network_profiles.yaml`, and `experiments.yaml`. Distinguish archived Day 1 settings from proposed FL settings.

[S7] Source and tests: `src/preprocess.py`, `src/centralized.py`, `src/metrics.py`, the remaining FL scaffold files under `src/`, and files under `tests/`. Exact source inspection establishes implementation versus placeholder status.

[S8] Artifacts and migration: the dataset, preprocessing, and relocation JSON files under `data/manifests/`; saved preprocessing/training artifacts; and the migration record in `paper/day1/MIGRATION.md`. These explain historical paths, local resolution, fingerprints, and prior test counts.

[S9] Supervisor audit: `paper/docs/Audit-DSIC-2907-Revisi.pdf`. Its inspected opening sections support the shared asynchronous backbone, two-control scope, catalog adaptation, and cautious novelty framing.

[S10] Manuscript: `paper/manuscript.md`. Contains the planned contribution and outline, not completed empirical results. Existing local edits were included in the inspected snapshot.

[S11] Package contract: `pyproject.toml`. Declares Python compatibility, dependencies, packaging, and lint/test settings.

[S12] Checks performed for this guide: the following record, produced on 3 October 2026 against the inspected working tree.

Actual verification record

Check	Outcome
Repository identity	Root `/home/fadil/DSIC-2907`; HEAD begins `c799c8e`.
Initial working-tree state	Existing modifications in manuscript and technical plan; preserved.
Code index	Project absent; indexing worker failed. Exact files were inspected directly.
System-environment tests	Collection blocked by missing pandas.
Tests in pre-existing TA environment	54 passed in 7.36 seconds.
Ruff on `src tests`	One existing F841: unused `staleness` in async server.
Saved artifact checks	Eight linked artifact fingerprints matched.
Model-result handling	Read archived JSON; no retraining or official-test reevaluation.

Commands used for the successful test and lint checks:

```
rtk proxy /home/fadil/TA/Code/demand-aware-async-fl-nids/.venv/bin/python \
  -m pytest -q -p no:cacheprovider
rtk proxy /home/fadil/TA/Code/demand-aware-async-fl-nids/.venv/bin/python \
  -m ruff check src tests
```

The fingerprint check called the existing `verify_ref` resolver for the result's checkpoint, configuration, preprocessing manifest, dataset manifest, and the preprocessing manifest's four artifacts. No raw-source revalidation or package build was performed for this document task.

The base commit is:

`c799c8eef29b493b2479145b2546421d9a9c1058`

It identifies the base repository snapshot, not the original centralized training run. Because local documentation changes were present, the commit alone does not reproduce every inspected document word.

Infrastructure-planning revision record

The 5 October 2026 addition updates the proposed infrastructure and its ownership questions. The original verification record above remains historical; its test, lint and fingerprint results were not rerun for this documentation revision. The revision does not certify RunPod capabilities or change the implementation-status claims. Revision checks: Pandoc/XeLaTeX compilation succeeded; section numbering and extracted PDF content were checked; rendered infrastructure pages were inspected for table clipping, diagram overflow and page breaks. No research tests, RunPod deployment or provider-capability validation were performed for this revision.

Day 2 verification record — 6 October 2026

The eight saved Day 1 artifact fingerprints were checked again. Partition generation verified the official training source hash and reused the frozen pipeline and split. For each configured partition seed (42, 123, 2026), all 140,272 train-fit rows occur exactly once across ten clients, validation is

excluded, per-client class counts match the training labels, and a fresh-directory replay reproduces identical partition/CSV bytes. Manifest content also matches after accounting for absolute output paths; same-path replay is covered by tests. The official test CSV was not opened and no model was retrained.

The full suite passed **75 tests**. The 21 new partition tests cover coverage, determinism, invalid inputs, global RNG isolation, single-class/empty-client handling, held-out exclusion, missing official test CSV, no preprocessing refit, tampered split rejection, overwrite refusal, artifact-byte replay, and multi-seed CLI generation. Day 2 lint and format checks passed; wheel and source distribution built successfully. Full-repository Ruff still reports only the existing F841 at `src/async_server.py:9`. Exact inputs, hashes, commands, and algorithm rules are recorded in `data/partitions/README.md`.

Day 3 verification record — 6 October 2026

The seed-42 CPU pilot used ten clients at alpha 0.5, three concurrent worker processes, one local epoch, batch 64, Adam 0.001, dropout 0.2, fp32, and one CPU thread per process. Dispatch was uniform among idle clients, and every update used budget 1.0. The recorded pilot equation is `global += (client_samples / 140272) * delta`; server learning rate and fixed staleness multiplier were both 1.0. These are pilot choices, not the main-experiment freeze.

Thirty accepted updates produced version 30 and 90 download/arrival/apply events. Client 0 downloaded version 0 and was applied at version 14, after 13 other applies. Validation Macro-F1 progressed through 0.242906, 0.736445, 0.920266, and 0.924870 at versions 0, 10, 20, and 30. The official test CSV was not opened. Arrival replay verified every message hash and reconstructed tensor-identical final model weights. OS scheduling can change the arrival order of a fresh live run; replay uses the saved trace, while local-training determinism is tested separately.

The full suite passed **93 tests**. Full Ruff lint, Day 3 format checks, and wheel/source builds passed. The former unused-staleness error is resolved by the implemented server receipt. The pilot records serialized tensor-payload lengths, excluding IPC/network framing overhead; complete byte accounting remains Day 5. Settings, commands, source hashes, artifacts and limitations are recorded in `paper/docs/Jurnal/Day-3-B0-Asynchronous-Training.md` and `results/processed/b0_day3_seed42.json`.

Updating this guide

Keep the Markdown source with the PDF. When Day 2 or later work is actually completed, update the status table only after its artifacts and checks exist. Retain the distinction between planned values and frozen values. Replace example outcomes with measured outcomes only when the run evidence supports them.

The PDF can be rebuilt using the already available Pandoc and XeLaTeX tools:

```
rtk proxy pandoc paper/TA-Ownership-Guide.md \
  --pdf-engine=xelatex --toc \
  -o paper/TA-Ownership-Guide.pdf
```

18. Plain-English glossary

Term	Meaning in this TA
TA	Your final academic research project; this guide concerns DSIC-2907.
NIDS	A system that detects suspicious or attack-related network activity.
Binary classification	Choosing between two labels: Normal and Attack.
Feature	An input measurement or category used by the model.
Label	The known answer used for learning or evaluation.
MLP	A neural network built from layers of connected units.
Epoch	One pass over the training examples available to that training process.
Batch	A group of examples processed together during training.
Checkpoint	Saved model weights and associated run information.
FL	Federated Learning: separate clients help train a shared model.
Logical client	A simulated participant; it need not be a separate physical computer.
Asynchronous	Arriving updates can be handled without waiting for everyone.
Aggregation	The rule used to incorporate client information into the global model.
Model delta	The change between a client's starting and trained model.
Non-IID	Clients have different statistical distributions of data.
Dirichlet alpha	A setting controlling how uneven the simulated class allocation tends to be.
Seed	A recorded starting value for repeatable random operations.
State	Current or recently observed information about a client.
Demand	Constructed scores that guide participation and update-budget decisions.
Actuator	A setting the controller can change; here, participation or update budget.
Cooldown	A required wait before a client becomes eligible again.
Staleness	How many server model versions passed since a client's starting model.
Starvation	A client repeatedly fails to receive a useful chance to contribute.
Top-k	Keeping the k entries with the largest absolute changes.
Serialization	Converting model information and metadata into transferable bytes.
Metadata	Descriptive information needed to interpret a message or artifact.
RTT	Round-trip network delay.
Throughput	How much data is transferred per unit of time.
Train-fit	Rows allowed to teach preprocessing and the model.
Validation	Held-out development data used for permitted checks and selection.
Official test	Held-out final evaluation data, protected from iterative tuning.
Precision	Among examples predicted as Attack, the fraction actually Attack.
Recall	Among actual Attack examples, the fraction detected.
Macro-F1	Mean of the two class F1 scores, weighting both classes equally.
Average Precision	A summary of precision/recall across score thresholds; the project's AUPRC implementation.
False positive	Normal traffic incorrectly predicted as Attack.
False negative	An attack incorrectly predicted as Normal.
Ablation	Removing or isolating a component to determine its contribution.
Paired comparison	Comparing policies under matching seed and experimental conditions.
Non-inferiority	Checking that quality is not worse beyond an allowed margin.

Term	Meaning in this TA
Confidence interval	An uncertainty range produced by a specified statistical procedure.
Pareto trade-off	A situation where improving one objective may worsen another.
Censored/not reached	The target was not observed before the run's stopping limit.
Manifest	A record identifying data or artifacts and their relevant properties.
SHA256	A content fingerprint used to detect changed or mismatched files.
Provenance	The trace from a result back to its data, settings, and artifacts.
Protocol freeze	Recording the rules before the main results can influence them.
Quality gate	Required evidence before a stage is considered complete.

Keep this sentence in mind: your job is to show whether the two communication decisions help, why they help or fail, and exactly what evidence supports that conclusion.